

Lista de Desafios: Testes de Software

Cinco desafios progressivos para praticar testes automatizados (a linguagem é sua escolha)

Disciplina: Teste de Software (8º semestre)

Professor: Ryam Soares da Silva

1. Tema e objetivo

Esta lista traz cinco desafios práticos para você exercitar a escrita de **testes automatizados** sobre funções que você mesmo cria. Você já sabe programar: agora o foco é garantir, com testes, que o código faz o que promete. A proposta é que você **pesquise** os recursos do seu framework de testes para resolver cada desafio, exatamente como acontece no dia a dia de quem desenvolve. Os desafios estão organizados por nível: **dois fáceis, dois médios e um difícil**.

Objetivo: aplicar na prática os conceitos vistos nas aulas introdutórias (o que é testar, casos de teste, asserções, o ciclo do teste) escrevendo funções e seus respectivos testes, e desenvolver autonomia para buscar soluções na documentação do seu framework.

2. Como funciona e entrega

- Escolha a sua linguagem e framework de testes: **pytest** (Python), **Jest** (JavaScript) ou outro de sua preferência (JUnit, Go testing, PHPUnit, etc.). Use a mesma escolha em todos os desafios.
- Resolva os desafios na ordem, do mais fácil ao mais difícil.
- Coloque cada desafio em sua própria pasta (**desafio1**, **desafio2**, ...), com o arquivo da função e o arquivo de teste separados.
- Todo teste deve rodar com um único comando (por exemplo, **pytest** ou **npx jest**) e passar. Inclua no **README** como instalar e executar.
- Muitos recursos ainda não foram vistos em aula: **pesquise!** Cada desafio traz dicas do que procurar.
- Trabalho **individual**. Publique tudo no seu repositório do **GitHub** e envie-me o link. Prazo: até a data combinada em sala.

3. Os desafios

Desafio 1 - Primeiros testes de uma função [Fácil]

Escreva uma função simples à sua escolha (por exemplo, converter Celsius para Fahrenheit, calcular a área de um retângulo ou verificar se um número é par) e escreva testes que confirmem o resultado esperado para pelo menos três entradas diferentes.

- **Função:** uma função pequena e pura, que recebe valores e devolve um resultado, sem depender de nada externo.
- **Teste:** pelo menos três casos verificando a igualdade entre o resultado e o valor esperado.

Pesquise: como instalar e rodar o framework; a estrutura básica de um teste (**def test_** no **pytest**, **test()** no **Jest**); a asserção de igualdade (**assert ==** e **expect().toBe()**).

Desafio 2 - Casos de borda e valores-limite [Fácil]

Crie uma função que decide algo com base em faixas de valores (por exemplo, a situação do aluno pela nota, a categoria de um IMC ou a faixa de preço de um frete). Escreva testes focados nos limites das faixas, onde os defeitos costumam se esconder.

- **Função:** retorna um resultado diferente conforme faixas de entrada.
- **Teste:** casos exatamente no limite e logo acima e logo abaixo de cada faixa.

Pesquisa: análise de valor-limite (**boundary value analysis**); como escrever vários casos de teste; como ler a mensagem de falha quando um teste quebra.

Desafio 3 - Testando erros e exceções [Médio]

Escreva uma função que deve recusar entradas inválidas lançando um erro (por exemplo, sacar mais que o saldo, dividir por zero ou tirar a raiz quadrada de um número negativo). Teste tanto o caminho feliz quanto os casos que devem lançar erro.

- **Função:** valida a entrada e lança uma exceção com mensagem clara quando ela é inválida.
- **Teste:** um caso válido e pelo menos dois que verificam se a exceção é lançada.

Pesquisa: como testar exceções (**pytest.raises** no pytest, **expect(...).toThrow()** no Jest); boas mensagens de erro.

Desafio 4 - Muitos casos com testes parametrizados [Médio]

Crie uma função de validação (por exemplo, validar uma senha forte, um e-mail simples ou um CPF com formato correto). Em vez de repetir o mesmo teste várias vezes, use testes parametrizados para cobrir muitos casos de uma vez. Ao final, encontre uma entrada que passa no seu teste mas que ainda representa um problema, e comente por que isso ilustra que um teste que passa não prova a ausência de defeitos.

- **Função:** recebe um valor e devolve verdadeiro ou falso conforme a regra.
- **Teste:** uma tabela de casos (válidos e inválidos) usando parametrização.

Pesquisa: testes parametrizados (**@pytest.mark.parametrize** no pytest, **test.each** no Jest); partição de equivalência.

Desafio 5 - Isolando dependências com test doubles [Difícil]

Escreva uma função ou classe que dependa de algo externo (por exemplo, um carrinho que consulta um serviço de preços, uma função que usa a data atual ou algo que leia de um repositório). Teste essa lógica sem chamar a dependência de verdade, substituindo-a por um duplê de teste (mock ou stub). Use também uma preparação comum a todos os testes (setup) para não repetir código.

- **Função:** a lógica principal recebe a dependência de fora (injeção de dependência), o que torna possível substituí-la no teste.
- **Teste:** um mock ou stub no lugar da dependência real e um setup (**fixture** no pytest, **beforeEach** no Jest) compartilhado.

Pesquisa: duplês de teste (mock, stub, spy); **unittest.mock** no pytest, **jest.fn()** e **jest.mock()** no Jest; setup e teardown; noção de cobertura de código (**coverage**).